

GAZEL: Runtime Gaze Tracking for Smartphones

Joonbeom Park, Seonghoon Park, Hojung Cha
Department of Computer Science
Yonsei University
Seoul, Republic of Korea
{jb.park, park.s, hjcha}@yonsei.ac.kr

Abstract—Although work has been conducted on smartphone gaze tracking, the existing techniques are not pervasively used because of their heavy weight and low accuracy. Our preliminary analysis shows that these techniques would work better if their models were trained with data from tablets which have large screens. In this paper, we propose GAZEL, a runtime smartphone gaze-tracking scheme that achieves high accuracy on real devices. The key idea of GAZEL, a tablet-to-smartphone transfer learning, is to train a CNN model with data collected from tablets and then transplant the model to a smartphone. To achieve the goal, we designed a new CNN-based model architecture that is head pose resilient and light enough to operate at runtime. We also exploit implicit calibration to alleviate errors caused by differences in users' visual and device characteristics. The experiment results with commercial smartphones show that GAZEL achieves 27.5% better accuracy on smartphones compared to the state-of-the-art techniques and provides gaze tracking at up to 18 fps which is practically usable at runtime.

Index Terms—runtime, gaze tracking, mobile, smartphone, transfer learning, head pose

I. INTRODUCTION

The two-dimensional (2D) gaze-tracking technique that identifies where the user is looking on a display has actively been investigated because of its variety of application domains. The applications include a computing interface for patients with general paralysis [1], user-customized content recommendation (news, videos, and advertisements) [2, 3], equipment in medical studies [4, 5], and others. Recently, owing to the widespread use of smartphones, efforts have been made to enable 2D gaze tracking on smartphones for mobile human-computer interaction (HCI) [6] and visual analytics [7, 8]. However, existing applications often require additional hardware for gaze tracking, which limits the mobility of smartphones resulting in degradation of the user experience.

To enable practical 2D gaze tracking on smartphones without additional hardware, machine learning-based techniques have been proposed [9, 10]. Using the built-in camera on smartphones, these works mostly focused on the large head pose variance in the image from the camera, which is the main cause of accuracy degradation. In addition, previous works rarely implemented a whole gaze-tracking framework; instead, they simply confirmed that the model architectures are computationally light enough to operate at runtime without validating the functionality with real device operations. We conducted a preliminary experiment to analyze whether the

existing smartphone gaze-tracking method works adequately in a real environment. We observed that the model trained with data collected from a smartphone (hereafter, the smartphone data model) has lower coverage than that of the model trained with data collected from a tablet (hereafter, the tablet data model). One possible explanation is that training the smartphone data model is practically hard because the range of pupil movement is small due to the small screen size. However, the issue is alleviated with a tablet where a relatively large screen induces more movements in a pupil. Second, the smartphone data model was observed to be vulnerable to changes in the head pose. Although recent studies [9, 10] have tried to handle head pose variance by inferring the relative position and distance between the device and the user, they have not overcome the limitation of the gaze data collected from smartphones.

In this paper, we propose GAZEL (GAZE Learning), a framework that performs gaze tracking effectively on a smartphone at runtime. The key idea is tablet-to-smartphone transfer learning. The proposed scheme collects data from a tablet and transplants the trained model to a smartphone to overcome the limitations of gaze data collected from smartphones. This approach concentrates on expanding the coverage by training a tablet data model and subsequently increases the accuracy through implicit calibration. First, when training the tablet data model, we use a new convolutional neural network (CNN) based gaze estimation model that is resilient to various head poses. Our insight is that the pupil movement is reflected differently relative to where the center coordinate of the face is located on the screen. The gaze estimation model is trained offline on the server with data collected from a tablet. Second, when transplanting the tablet data model to a smartphone, we propose an implicit user data collection method and a calibration method to reflect the difference between the two devices and the user's personal characteristics. As the conversion process of the devices' screen coordinate system is linear, we use linear regression (LR) to reflect inter-device differences and inter-personal differences. For the LR training, we collect gaze data in the background whenever the user touches the application (app) icon in the launcher. This process is completely implicit for enhancing user satisfaction. Overall, our work is, to the best of our knowledge, the first approach to implement an end-to-end gaze-tracking framework which runs on off-the-shelf smartphones at runtime.

II. BACKGROUND AND RELATED WORK

A. Gaze Estimation Methods

Gaze estimation methods are divided into model-based or appearance-based [11]. Model-based approaches use a geometric eye model for estimating the gaze, and this approach

This work was supported by Next-Generation Information Computing Development Program funded by the Ministry of Science and ICT (Grant No. NRF-2017M3C4A7083677), National Research Foundation of Korea(NRF) (Grant No. NRF-2019R1A2C2004619), and Institute for Information & communications Technology Promotion(IITP) grant funded by the Korea government(MSIT)(No. 2018-0-00532, Development of High-Assurance ($\geq$ EAL6) Secure Microkernel).

is further divided into two groups: corneal reflection-based and shape-based methods. The corneal reflection-based methods exploit the glint (reflection) of external light sources on the user's cornea to estimate the gaze [12]. To overcome the obvious limitation of requiring an external light source to produce glint on the user's cornea, ScreenGlint [13] uses the screen reflection on the cornea to estimate the gaze on a smartphone. This approach still poses a problem in determining whether the glint belongs to that of a smartphone or other light sources. Shape-based methods [14, 15] use eye shapes, such as pupil centers and iris edges, to estimate the gaze point. Moreover, corneal reflection- and shape-based approaches suffer from variable lighting conditions and low image quality.

In contrast, appearance-based methods [16-18] directly use images as input for gaze estimation. Today, most of the appearance-based methods [9, 19, 20] utilize deep learning to increase the accuracy of the gaze estimation model. Appearance-based gaze estimation techniques that use CNNs have surpassed classical methods for in-the-wild settings [19]. These recent approaches, however, require a large amount of training data and significant computation power compared to regression methods, such as ridge regression (RR) [21] and support-vector regression (SVR) [22].

B. Implicit Calibration

Most gaze estimation models require calibration and recalibration to enhance accuracy according to changes in lighting conditions and head poses [23]. Calibration can be divided into explicit calibration and implicit calibration depending on the user's involvement. Explicit calibration requires a time-consuming and cumbersome calibration task that instructs the user to look at a specified point whenever recalibration is needed. Implicit calibration, however, collects data from the background and performs calibration without the user recognizing the process. Conventionally, various methods were considered to apply implicit calibration using image saliency [24], known fixation points [25], and others' previously collected calibration data [26]. Because of the accuracy issue and the necessity of additional hardware of the scheme, however, a new calibration method emerged, focusing on user-device interaction. Researchers [27] found that the gaze and the cursor have a close correlation within certain conditions. Huang et al. [28] discovered the "cursor lags behind gaze" phenomenon in which there is a delay between the actual gaze and the cursor. Huang et al. [29] further analyzed this phenomenon. For mobile devices, Weill-Tessier et al. [8] showed a similar correlation in gaze and touch using a tablet and the Tobii eye tracker [30].

C. Related Work

Shape-based gaze estimation has been investigated on smartphones [31, 32]. These studies used the smartphone simply as an interface to use the camera, not for gaze tracking on the screen. These shape-based methods have limitations in situations where the head poses and lighting conditions may change constantly. Studies have estimated the on-screen coordinates of a smartphone with an appearance-based approach suitable for an in-the-wild setting. Several studies tried to provide an accurate gaze estimation model resilient to the head pose variance and various lighting conditions. Using the Rice data set [18], TabletGaze [18] found that facial visibility is poor

(30%) in the wild environment, whereas the eyes are visible in most cases (96%). According to this finding, using only eyes as the input for gaze estimation is more applicable for runtime gaze tracking. However, this single approach is unlikely to handle head poses successfully, resulting in relatively low accuracy. Several approaches have been proposed to overcome the head pose issues. First, iTracker [9], unlike TabletGaze, uses four crops (left and right eye images, face image, and face grid) as input. The face grid is expected to represent the relative distance between the device and the user. The CNN-based AlexNet [33] was utilized to extract features from the left eye, the right eye, and face images. Using GazeCapture [9], iTracker claims to have achieved accurate calibration-free tracking. iTracker claims that the vast diversity of head poses and illuminance in the dataset will automatically make iTracker resilient to illuminance and head pose variance. Next, GazeEstimator [10] exploits the same approach as TabletGaze to use only eyes-related data as input and utilize the CNN-based ResNet for extracting features. GazeEstimator includes an eye grid as an alternative to the face grid in iTracker for handling head pose variance.

However, the existing approaches have limited accuracy and runtime use. First, iTracker and GazeEstimator do not cover the head pose variance that occurs in the wild adequately resulting in low accuracy. This is due to the data collected from the smartphone (we discuss the details in Section III). Second, there is a problem with the Rice data set and GazeCapture when collecting the datasets. It is important to collect data by considering the variance of the head pose and the use cases of smartphones. Normally, users do not use smartphones and tablets with their faces turned to the left or right or their faces rotated counter-clockwise or clockwise. Furthermore, it is uncommon for users to use smartphones with their arms raised above their heads. GazeCapture contains many of the outlier head poses. The study instructed participants to continuously move their heads and the distance of the phone relative to them to gather a random head pose and periodically change the mobile device's orientation. This method leads participants to gaze at the device while consciously changing to new and varied head poses, creating significantly different cases from that of the device used in the wild. In contrast, the Rice dataset provides the data as videos; in this way, the frame data are deteriorated to a certain degree due to compression when saving. This approach differs from obtaining a raw frame or bitmap from a camera stream in an actual device. In short, GazeCapture and the Rice data set, which do not reflect the runtime environment properly, are not suitable for model training. Unlike them, we have to examine how users behave when using a smartphone in a real environment and collect data accordingly. Last, more importantly, neither of the previous models can operate on a smartphone at runtime because of their complicated models. iTracker assumes that leveraging Apple's face detection pipeline will satisfy 10-15 frames per second (fps) on smartphones, but to meet this claim, iTracker must be optimized first to operate on smartphones at runtime. In fact, iTracker does not describe how to reduce the structure of a large model other than reducing the size of the input.

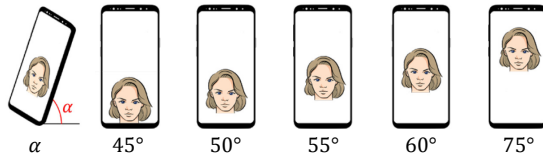


Fig. 1. Head pose relative to each angle α .

III. PRELIMINARY EXPERIMENTS

We conducted preliminary experiments to understand the characteristics of smartphone gaze estimation models and to present the need for a new model.

A. Experimental Setup

To collect gaze data, we used the latest devices, the Samsung Galaxy S9 Plus and the Samsung Galaxy Tab S6 tablet. On these devices, we collected data from ten participants (two women and eight men); four wore glasses. The age of the participants ranged from 25 to 35 years old. Data were collected only in portrait mode, taking into account the camera positions of both devices. We wrote an application to obtain data from the participants, using the Android Firebase ML Kit [34]. Using the ML Kit, we parsed eye and face images from the front-camera image at a resolution of 480×640 pixels in runtime. With face contour and landmarks, we constructed inputs for the two models and additional data for developing a new model. The data consist of a face and each eye grid, a ground truth gaze point (the center position of a button), each eye size, and the center point of the detected face. The application divides the device screen into 10×20 grids, and participants need to click randomly displayed buttons for data collection (one out of 200 grids).

We carefully designed the data acquisition process for participants. Most importantly, we considered the change in the head pose due to the change in the angle between the face and the mobile device. From Huang et al. [18], we inferred that the face appears mostly on the lower part of the front-facing camera image. The face appearing in the lower part of the image is caused by the angle between the face and the mobile device. In general, as the angle between the two increases, the face gradually descends to the bottom of the camera image. On the contrary, the face is rarely located at the top of the image. The device must be intentionally leaned toward the user so that the face is located at the top of the image, which is a very uncomfortable posture for users. This is because the camera is located at the top of the screen on most mobile devices. To consider these various head poses, we set the angle between the ground and the mobile devices at 45° , 50° , 55° , and 60° . Fig. 1 shows the head poses which are related to each angle. Data were collected at a dynamic distance (40-50 cm) such that the participant felt comfortable at each angle. We also included a 75° case in the data to cover extreme cases when the face was almost in the top region of the image. We also asked participants to move their head to the left or right side of the screen but not outside the screen to include additional head pose variance in the data.

When collecting the data, participants were instructed to stare at the clicked button until the button moved to another grid. While participants were staring at the button, the app started processing the camera image right after the click and stopped when the button moved. In this setting, the button moved its

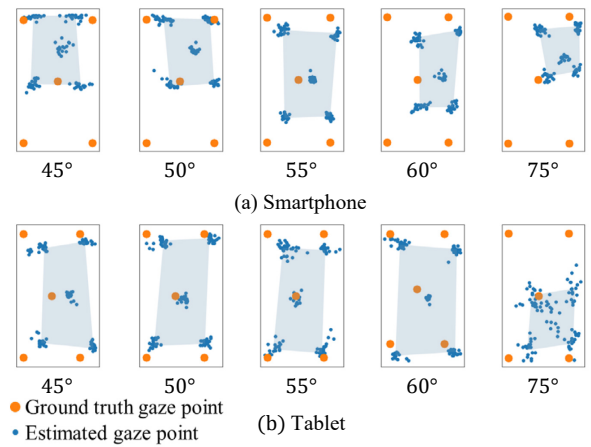


Fig. 2. Coverage difference in GazeEstimator trained with data collected from a smartphone and a tablet.

position 1 sec after the click. To avoid blinks in collected data as much as possible, we instructed the users not to blink after the click until the button changed position. To minimize the difference between the data collected from the smartphone and the tablet, we aligned the devices so that the position of the camera of the two devices was the same. The app stores the parsed data if the face is detected. The app processes 8-12 frames per sec on both devices, which means it is designed to save 8-12 data with one touch. We asked participants to collect at least 1000 samples of data for each head pose, which totaled at least 5000 samples per participant, taking about 30 min in total. We collected a total of more than 50,000 samples on each device.

B. Results

Fig. 2 shows the results of the estimated position of the user gazing at the four corners and the center of the screen in a real user environment with GazeEstimator. We provide only the case of GazeEstimator because iTracker showed similar results. Fig. 2(a) shows the experimental results for the smartphone data model. The four corners indicate the endpoints of the smartphone screen. Fig. 2(b) shows the results on the tablet with the tablet data model. The four points on the tablet case, however, do not indicate the four endpoints of the tablet screen. These four points are physically identical to those of the smartphone's four corners when the smartphone is placed on top of the tablet aligned by the camera position. The experiment showed that the smartphone data model has lower coverage than the tablet data model. The smartphone data model learns only the data on the smartphone screen; thus, the model incompletely learns regions close to the edge of the smartphone. In other words, due to the limitation of a small screen, only a small range of pupil movements are learned, resulting in low coverage. However, a model trained on a tablet with a larger screen learns a larger range of pupil movements. Therefore, it is possible to predict the location easily corresponding to the corner of the smartphone, which is a relatively small area.

In addition, Fig. 2(a) shows that the smartphone data model is vulnerable to head pose variance. The 45° and 50° cases showed almost the same results because the change in head pose was insignificant compared to other degrees. The 45° and 50° cases showed a poor estimation in the lower area of the smartphone. Note that larger the angle between the face and the

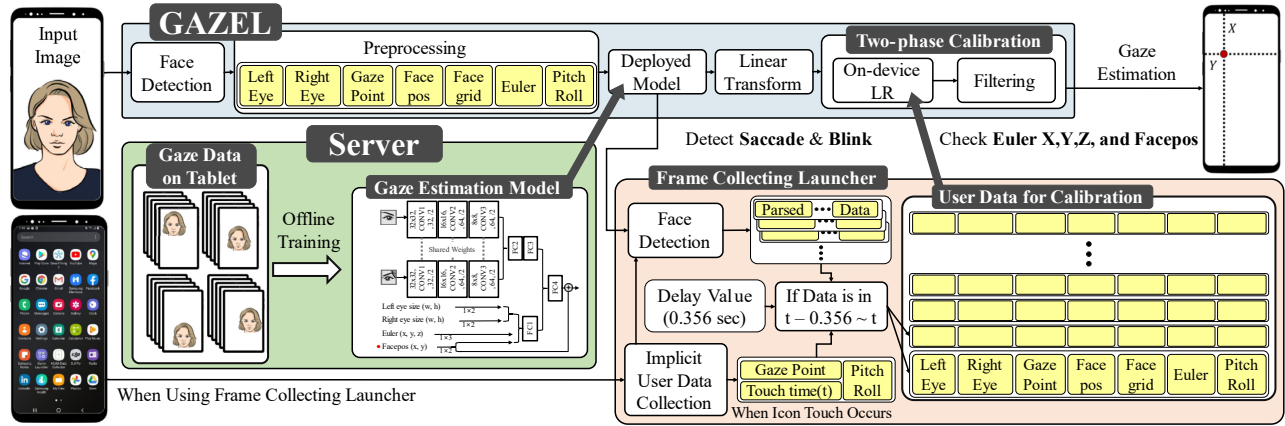


Fig. 3. GAZEL overview.

phone, the less the eye's vertical movement appears in the camera's image. The lower area is not well predicted because, in the 45° and 50° cases, the angle between the face and the phone is the largest among all of the cases. Meanwhile, 75° shows the lowest coverage. In this case, the distance between the user and the camera is the smallest, which leads to the worst result. Note that the pupil's movement appears smaller in the image when the distance between the user and the camera becomes smaller. However, 55° and 60° show relatively high coverage. In these cases, the face and the phone are positioned close to parallel, and the face is positioned near the center of the screen. Thus, the pupil movement appears relatively large.

In summary, the preliminary experiments showed that iTracker and GazeEstimator do not reflect the variance of the head pose adequately, when trained with data collected from smartphones. As these approaches do not reflect changes in the head pose adequately, coverage is lowered, subsequently resulting in poor accuracy overall.

IV. GAZEL DESIGN

A. GAZEL Overview

To solve the issues addressed above, we propose GAZEL. Fig. 3 shows the overall architecture of GAZEL. The key idea is tablet-to-smartphone transfer learning, which transplants the trained model with tablet data to the user's smartphone. The whole process of transfer learning is divided into the process of collecting data and training a gaze estimation model, followed by the process of transplanting the trained model to a smartphone for tracking gaze at runtime. When designing these two processes, we specifically consider that GAZEL should be light enough to operate at runtime while achieving high accuracy on various head poses.

The training process of the gaze estimation model starts by collecting gaze data for various people and head poses on a specific tablet, and ends with training the model with offline learning on the server. By constructing a device-specific model in this way, a highly accurate model can be provided to the smartphone. This gaze estimation model is based on a CNN-based deep learning model, and the structure of the AlexNet convolution (CONV) layer was modified in a lightweight manner for left and right eye feature extraction. In addition to these extracted features, input features were added to reflect the

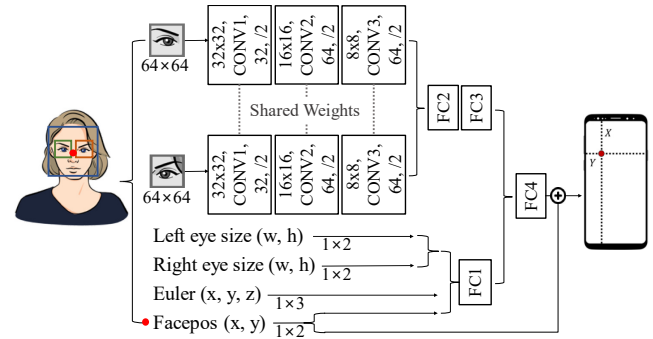


Fig. 4. Gaze estimation model.

head pose variance. The trained model is deployed as an app on various types of smartphones and used for inference.

As shown in Fig. 3, GAZEL's inference consists of several processes. Face detection is performed on the image captured by the smartphone's front-facing camera to find the location of the face and eye contours in the frame. Based on this, the input image is preprocessed and divided into several crops, which are used as the input of the trained model. These inputs are used by the deployed gaze estimation model to estimate the current gaze point. As the model's estimated point is a point adjusted to the tablet coordinates, we translate the point to the smartphone coordinates. Because this translated point is not practical for use, accuracy should be improved through the two-phase calibration. The first phase of the calibration process uses implicit user data collection and on-device LR to adjust the inter-personal differences and hardware differences. Implicit user data collection is performed through the proposed Frame Collecting Launcher in Fig. 3. For the second phase, a filtering technique is applied to the LR output to remove noise that continuously occurs during the runtime gaze-tracking process.

B. Model Training and Transplanting

The existing deep learning models for smartphone gaze tracking are too complicated to run at runtime, so we designed a new model that is lightweight. Fig. 4 shows the model structure that enables GAZEL to operate at runtime on real devices. We designed the model to increase accuracy by reflecting various head poses while operating quickly and lightly on smartphones.

We propose a new input layer that improves the accuracy of the model while lowering the computing overhead. When

designing the input layer, we modified the existing deep learning model by removing inputs that could adversely affect the accuracy and adding new inputs that could enhance the accuracy. The inputs of the new model architecture are left/right eye images, left/right eye sizes, Euler angles x , y , z , and facepos meaning the center point of the face. Unlike the iTracker model, we do not adopt the face images as input because the face crop images include the faces' outer areas, which could negatively affect the accuracy. The most important input, the left/right eye images, is an image that crops only the eye area from the entire image. Similar to the method used by Zhang et al. [19], the eye image was extracted to be 64×64 pixels so that the computation overhead was reduced in the process of extracting features. Unlike other smartphone-based gaze estimation models [9, 10], we used grayscale images. This is because the accuracy is higher when using grayscale (one channel) images than RGB (three-channel) images. In addition, left and right eye sizes replace ambiguous left and right eye grids consisting of 0s and 1s, which were used to learn the relative distance difference in the existing model. These left and right eye sizes are expressed as the width and height relative to the entire frame. Euler angles indicating the orientation of the head and facepos indicating the position of the center point of the head in the entire image are inputs added to better reflect the head pose.

We designed the layer structure to contain features that reflect the head pose while being lightweight. Head pose variance is reflected by combining three factors (facepos, Euler angles x , y , and z , and eye sizes) into a fully connected layer. The number of extracted features of the two layers was adjusted before concatenating so that the features of the two eyes can be affected by the head pose feature. The highlight of the proposed model is the Add layer that adds the facepos at the last fully connected (FC) layer. Adding the Add layer, we force the model to express how far the gaze point is based on the facepos. This method allows the model to learn the eye movements with different ratios depending on the facepos. With this approach, we were able to create a lighter model with better accuracy than the existing methods that directly estimate the gaze point without a reference point. We use the following sizes for the CONV layers: CONV1 ($3 \times 3 \times 32$), CONV2 ($3 \times 3 \times 64$), and CONV3 ($3 \times 3 \times 64$). Between the convolution layers, we used a max-pooling layer (2×2). The FC layers are the following sizes: FC1 (16), FC2 (64), FC3 (16), and FC4 (2).

As the proposed model is trained on tablet data, we added an appropriate conversion layer to apply the model to the smartphone. We designed the conversion process as a linear transform layer. First, in the transform layer, the screen coordinate system of the source device is translated into the camera coordinate system because the camera's location is different for each device. Then, the point is scaled down according to the resolution and physical screen size difference between the source and target devices. Last, the camera coordinate system is transformed into that of the target device. Equations (1) and (2) explain the whole linear transformation process.

$$X' = \left(X - \frac{W_{original}^{pixel}}{W_{original}^{cm}} C_{original}^{xpos} \right) \frac{W_{original}^{cm}}{W_{target}^{pixel}} + C_{target}^{xpos} \frac{W_{target}^{pixel}}{W_{target}^{cm}} \quad (1)$$

$$Y' = \left(Y - \frac{H_{original}^{pixel}}{H_{original}^{cm}} C_{original}^{ypos} \right) \frac{H_{original}^{cm}}{H_{target}^{pixel}} + C_{target}^{ypos} \frac{H_{target}^{pixel}}{H_{target}^{cm}} \quad (2)$$

X and Y refer to the coordinates estimated from the tablet data model using the smartphone input data. X' and Y' refer to the screen coordinate values of the smartphone that have undergone a linear transformation. W_A^B denotes the A device's width in B , H_A^B the A device's height in B , and C_A^B the camera's position in centimeters relative to the B axis from the top left of the screen on device A . Through this process, the coordinates based on the tablet are converted into coordinates based on the smartphone. This process enables gaze estimation on a smartphone with high coverage.

C. Two-phase Calibration with Implicit User Data Collection

Although the proposed model improves coverage, calibration is still needed to increase accuracy while maintaining good coverage. The calibration must reflect the inter-personal differences and hardware differences, such as camera-intrinsic parameters and pixel density, when transplanting a tablet-based model to a smartphone. The key challenge is how to calibrate the model's output according to personal and hardware differences without tedious explicit calibration. We divided this implicit calibration process into two steps: implicit user data collection with linear regression and filtering.

The first step is calibrating inter-personal differences and hardware differences. To obtain these differences from a user at runtime, we use an implicit data collection technique that collects personal data without causing the user inconvenience (Implicit User Data Collection in Fig. 3). Similar to a previous study that implicitly collected data by a correlation between a cursor and a gaze using a mouse [27-29], we focus on the correlation between touch and gaze correlation on a smartphone. However, similar to a click [35], a touchpoint does not always correspond to the gaze point, because when a keyboard is used, there are many cases of typing by looking at the area where the typing occurs, not the keyboard. In addition, when the button is large, there is quite a gap between the touch point and the gaze point, such as when clicking the image or selecting a target from a list. Therefore, the correlation between the gaze point and the touch point should be considered carefully. In our work, we focused on the launcher, an application that keeps the button size small and must be consciously checked before pressing the button. We created a customized launcher application, Frame Collecting Launcher, to collect frame data whenever the user touches the app icon.

To collect user data, we need to consider the delay between a gaze and a touch, just like the delay between the cursor and the gaze [28]. According to a study on the delay between the gaze and touch on a tablet [8], the point gazed at 0.356 sec, on average, before a touch occurred is the most similar to the point touched. Therefore, we set 0.356 sec as the delay value in Fig. 3. We collect all the data when the launcher is used before touching occurs. When a touch occurs at time t , the app permanently stores the data from $t-0.356$ sec to t , and other data are deleted. However, saccade or blinking may occur during collection. The transported gaze estimation model filters this saccade or blinking data. For later calibration, the roll and pitch of the device at the time the data were collected are obtained using a

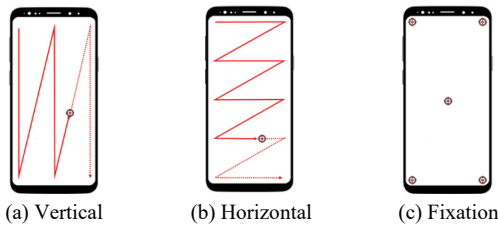


Fig. 5. Gaze target for evaluation.

rotation vector sensor. When the user executes the gaze tracker, the data stored are used for calibration, by extracting only data similar to that of the facepos, roll, and pitch of the current device. Using the extracted data, we used on-device LR, as the calibration of translated points needs linear transformation.

For the second calibration process, a filtering technique is applied to the LR output to continuously remove noise in the gaze-tracking process. In runtime gaze tracking, a fluctuation occurs inevitably according to noise generated by the camera or the surrounding environment. Due to this fluctuation, it is difficult to accurately extract the gaze movement in runtime. The exponential moving average filter is used to minimize the fluctuation. We chose the filter from various low-pass filters because it provides fast responsiveness and appropriate filtering. The exponential moving average in GAZEL is implemented using 0.4 for the coefficient α .

V. IMPLEMENTATION

We implemented GAZEL based on the Android Firebase ML Kit. Firebase face detection was used for runtime face detection, which gives us pretty much all of the data we need for gaze tracking. The tool provides the facepos, Euler angles, and eye contours needed to crop each eye and compute each eye size. After the eyes were cropped, Android RenderScript [36] was used to accelerate the process of converting RGB eye images to grayscale eye images. Using the Firebase ML Kit, GAZEL processes the input data in 18 fps on the Galaxy S9 Plus.

For the gaze estimation model, we used data collected on the Galaxy Tab S6 for training. We used Keras with TensorFlow 2.0 backend and Euclidean loss as a loss function like iTracker. The trained model's accuracy on the validation set was equal to 1 cm. The Keras .h5 file was converted to the .tflite format with Python API [37]. TensorFlow Lite Java API, which is available as the Android library (org.tensorflow: tensorflow-lite), was used to run the tflite model on a smartphone. The TensorFlow Lite Interpreter was initialized using the .tflite file with the provided API. The output shape was mapped to support two outputs of x and y, using HashMap. The runForMultipleInputsOutputs [38] method was used to infer the estimated gaze points with preprocessed inputs. The estimated points were converted using the linear transformation described in (1) and (2) to match the Galaxy S9 Plus screen coordinates because the coordinates of the gaze estimation model refer to the Galaxy Tab S6. For calibrating the estimated gaze point, we implemented LR. The two linear regressors were used for each x and y coordinate.

VI. EVALUATION

In this section, we evaluate the accuracy and overhead of GAZEL. First, GAZEL's accuracy is evaluated on the gaze estimation model and the two-phase calibration. Then, the

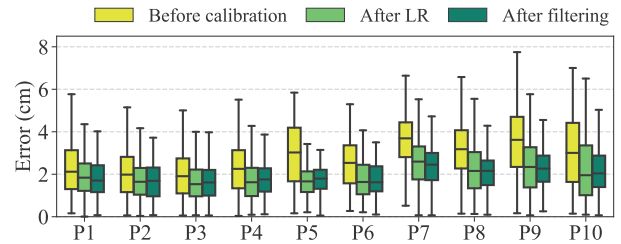


Fig. 6. Average accuracy for different participants.

runtime overhead for GAZEL is evaluated. Finally, the contribution of GAZEL is validated by comparing it with existing smartphone gaze-tracking techniques.

A. GAZEL Performance

1) *Experimental setup*: To evaluate GAZEL's accuracy, the experiment was carefully designed to test the gaze tracker on a smartphone. We conducted the overall process of GAZEL with the same ten participants who participated in the preliminary experiment. To reflect the horizontal and vertical gaze movements, participants were asked to stare at the gaze trajectory shown in Fig. 5(a) and (b). The fixation experiment is also shown in Fig. 5(c). The various head poses shown in Fig. 1 were evaluated in the experiment, and data were collected in portrait mode. In the vertical case shown in Fig. 5(a), the test was designed to take 2 sec per straight line for the participant to gaze at. In the horizontal case shown in Fig. 5(b), it took 1 sec per straight line. In the fixation case in Fig. 5(c), participants were instructed to stare at each point for 2 sec; thus, the total time for all three cases was 9-10 sec. The time is long enough to extract a sufficient number of results from each line and short enough for the participants to gaze at the entire trajectory without blinking. We tried to eliminate blinking because blinks could adversely affect the gaze estimation test results. For collecting calibration data, participants were instructed to touch the app icons in the given order on the Frame Collecting Launcher explained in Section IV.

For the GAZEL runtime experiment, we wrote a smartphone application that allows participants to follow the trajectories in Fig. 5. This application displays the gaze target that the user should track on the smartphone screen and at the same time, estimates the gaze point of the user using the model. To show the improvement in accuracy step by step, we logged the raw results without LR, the results through LR, and the results through filtering. The tablet data model and the smartphone data model were ported to the Galaxy S9 Plus and tested with the same input at the same time.

2) *Accuracy of the overall process*: We evaluated the overall process of GAZEL with ten participants (P1-P10). Fig. 6 shows the average accuracy of all head poses before calibration, after LR, and after filtering for each participant. As shown in the figure, the accuracy is improved through LR for all cases and further enhanced through the filtering process for most cases. Checking the inter-personal differences, we found that the overall accuracy of the person wearing glasses (P7-P10) is low as the reflection from the glasses and the glass frame in the eye crop image hampers the model's ability to track gaze accurately. The female participants (P5, P9) also showed low

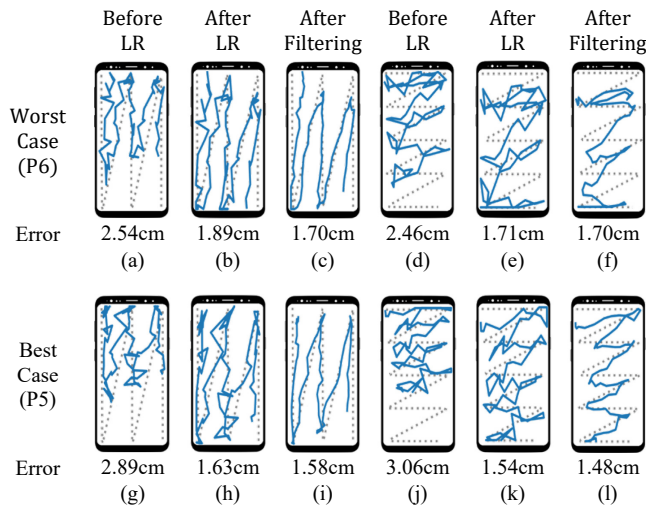


Fig. 7. Accuracy enhancement during two-phase calibration (55°).

accuracy, presumably due to the visual difference between men and women.

To see how each calibration process enhances gaze prediction, we visualized the interim results of the two-phase calibration process. Fig. 7 shows the worst case and the best case at 55°. Fig. 7(a-f) show the results for the worst case from P6, and the rest show the results for the best case from P5. Fig. 7(a-c, g-i) show the results when gazing at the trajectory of Fig. 5(a), which is a vertical case, and the rest are horizontal cases shown in Fig. 5(b). The results before LR is applied are shown in Fig. 7(a, d, g, j), after LR is applied with five points in Fig. 7(b, e, h, k), and after filtering is applied in Fig. 7(c, f, i, l). Overall, the results show that calibration is essential in runtime gaze tracking, so that the filtering follows the actual gaze precisely. In addition, the high accuracy before calibration does not necessarily ensure improved accuracy after the calibration process. In particular, the accuracy in Fig. 7(g) and (j), which was worse than Fig. 7(a) and (d) before LR, improved significantly after LR. This improvement continued in the subsequent filtering process, resulting in better accuracy after LR and filtering. Moreover, Fig. 7(g) and (j) contain little noise; thus, their estimated gaze trajectory can be displayed more distinctly after the overall process, leading to better accuracy. This finding contradicts the notion of existing studies which did not consider the calibration process important. However, these results suggest that the calibration process results are also important.

3) *Accuracy of the gaze estimation model*: To analyze the GAZEL's gaze estimation model, we evaluated the accuracy of the model in terms of various aspects without the calibration process. First, we evaluated how accurately the gaze estimation model works in head positions. Fig. 8 shows how accurately the tablet data model and the smartphone data model operate in various head poses on the Galaxy S9 Plus. The models trained with data collected on the tablet were shown to behave more accurately in all head orientations. This result, which reconfirms the results in Fig. 2, indicates that the proposed tablet-to-smartphone transfer learning technique has better coverage than the model trained only with smartphone gaze data and is resilient to various head poses.

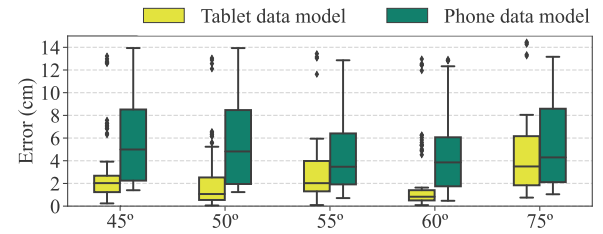


Fig. 8. Accuracy comparison between models trained on tablet data and smartphone data for different head poses.

TABLE I. ABLATION STUDY (TABLET DATA MODEL)

Model	Average Error for Various Head Poses (cm)
GAZEL	2.91
GAZEL (no Shared)	3.23
GAZEL (RGB image)	3.11
GAZEL (no Euler)	3.05
GAZEL (no eye size)	3.15
GAZEL (no facepos)	3.43

Next, an ablation study was conducted to evaluate each input parameter's contribution to the overall accuracy. In an ablation study, the model's input parameters are removed one by one, and how those parameters affect the model's overall accuracy is checked. The average accuracy values obtained in various head poses for each ablated model tested on the data collected in the previous section are summarized in Table I. When the shared weights in the CONV layers are removed, the accuracy is degraded. Removing the shared weights puts more emphasis on the difference than on the similarity of the eyes, which shows that this method is not advantageous for gaze estimation. The results also show that using RGB images instead of grayscale images as GAZEL input leads to a larger error. This suggests that the RGB images input is disadvantageous for gaze estimation. When converting an image consisting of R, G, and B channels into a grayscale image, three channels are converted into a single luma component. The grayscale images express illuminance inherently better, and this leads to improved accuracy over the use of RGB channels. On the other hand, when the Euler angles or eye size was ablated, there was only a slight decrease in the accuracy. Finally, ablating the facepos, which ablates the Add layer as well, leads to critical accuracy degradation because the proposed model structure has a small number of trainable parameters compared to other models. Through the ablation study, we confirmed that each GAZEL input component is necessary for the proposed model to be lightweight while achieving good accuracy.

4) *Effect of implicit calibration on accuracy*: We validated the overall accuracy in each phase of the two-phase calibration, i.e., linear regression and filtering. First, to check the improvement in the accuracy of gaze tracking with LR, we compared the LR with non-calibration, RR, and SVR. For this experiment, the change in accuracy during calibration was tracked when including calibration points one by one. There are a total of 35 calibration points as the Frame Collecting

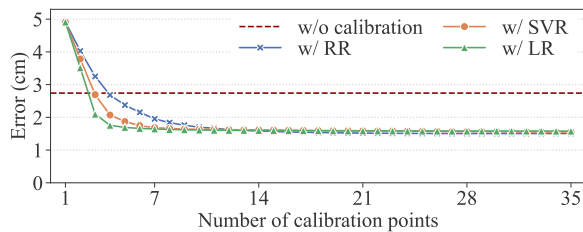


Fig. 9. Accuracy of regressions according to the number of calibration points.

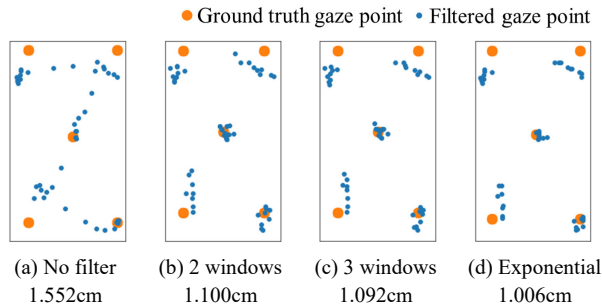


Fig. 10. Filtering methods and accuracy.

Launcher used for collecting user gaze data has a 5×7 grid layout. The experiment was repeated 100 times. Fig. 9 shows the improvement in accuracy when n calibration points randomly out of 35 calibration points are used for each method. All methods eventually become more accurate than the model without calibration. In addition, LR was shown to converge fast, compared to other calibration methods. However, accuracy is degraded if fewer than two calibration points are used for LR. The results suggest that LR-based calibration should be performed when at least three points are collected. The improvement in the accuracy slows from more than four points and converges to almost 1.5 cm, which indicates that this LR calibration is sufficiently effective with a few calibration points.

To find the effectiveness of the filtering technique, we compared the results of applying various filtering techniques to the calibrated gaze point described in Fig. 5(c). Fig. 10 shows the comparison results for non-filter, moving average with two or three windows, and exponential moving average. As shown in Fig. 10(a), even if linear calibration is used, the fluctuation in the estimation points is still severe due to the noise caused by the camera or the surrounding environment. This result indicates that a low-pass filter is needed to reduce the fluctuation. Fig. 10(b) and (c) show the result of a moving average method that uses the average of the stored previous gaze points as well as the current gaze point, as the filtered gaze point. With this method, the larger the window size, the more accurately the fixation point can be estimated. However, as the window size increases, the moving average reacts slowly to the movement of the pupil. The window size should be small because human eyes move quickly in a saccade. With a window size of three or more, the user may feel that the response is slow. On the other hand, the exponential moving average method reflects the previous estimation points and the current estimation point according to the coefficient α value. Fig. 10(d) shows the final results for GAZEL, α set to 0.4, which results in high reactivity while filtering fluctuations properly.



Fig. 11. Overhead on commercial smartphones.

5) *Overhead*: We evaluated the runtime overhead of GAZEL with commercial devices. To consider the performance characteristics of the application processor (AP) on the device, we conducted experiments on the Pixel XL, Pixel 2 XL, Pixel 3 XL, Galaxy S9 Plus, and Galaxy S20 Plus smartphones. The devices are listed in order of AP performance: The Pixel XL had the worst processor performance, and the Galaxy S20 Plus the best. Fig. 11 shows the time spent in GAZEL's overall process of gaze tracking on various devices. The time spent on linear regression and filtering was less than 1 ms; thus, the results are omitted in the figure. Similar to the iTracker study, we set 100 ms as the limit at which the model is considered to be operable at runtime. According to Fig. 11, the total latency is 57 ms (18 fps), 83.3 ms (12 fps), and 115 ms (8 fps) for the Galaxy S20 Plus, Galaxy S9 Plus, and Pixel 3 XL, respectively. These results indicate that GAZEL can work at runtime on smartphones with moderately fast processors. However, for devices with low computing power (the Pixel XL and Pixel 2 XL), the latency is much longer than 100 ms, meaning the user experience is degraded. As demonstrated in the figure, the face detection process significantly influences the latency; thus, the GAZEL framework is bottlenecked with the face detection overhead. If the basic framework for face detection improves, the overall latency of the gaze tracking will decrease further in the future.

B. Comparison with Previous Research

We compared GAZEL with two state-of-the-art gaze-tracking techniques developed for the smartphone environment, iTracker and GazeEstimator. Unlike GAZEL, the iTracker and GazeEstimator models are far too heavy to run on-device at runtime, making it impossible to conduct the test in the wild. Therefore, to test the models, we modified the application used in the runtime experiment for GAZEL in Section VI.A to collect participants' gaze data. The application extracts data from the front-facing camera image of the user staring at the gaze target. The extracted gaze data are then transferred to the desktop environment, used for testing the models offline.

The comparison results for GazeEstimator, iTracker, and GAZEL are summarized in Table II. GazeEstimator and iTracker collect data on smartphones and tablets for training, whereas GAZEL uses data only from a tablet. The model size of GazeEstimator and iTracker is at least 50 times larger than that of GAZEL. Due to the size, neither method can be ported and operate on an actual smartphone. In terms of the calibration process, GazeEstimator did not implement calibration, and iTracker performed SVR-based explicit calibration with 13 points, whereas GAZEL minimizes user discomfort by performing implicit calibration. Moreover, existing works did

TABLE II. COMPARISON OF MODELS AND AVERAGE ACCURACY ON SMARTPHONE

Model	Train Data	Size (KB)	Calibration Method	Filtering	Runtime	Before Calib Error (cm)	After Calib Error (cm)	After Filtering Error (cm)
iTracker[9]	Smartphone + Tablet	303,557	Explicit SVR with 13 points	X	X	3.73	1.90	X
GazeEstimator[10]	Smartphone + Tablet	122,150	X	X	X	3.86	X	X
GAZEL	Tablet	2,281	Implicit LR	O	O	2.91	1.58	1.49

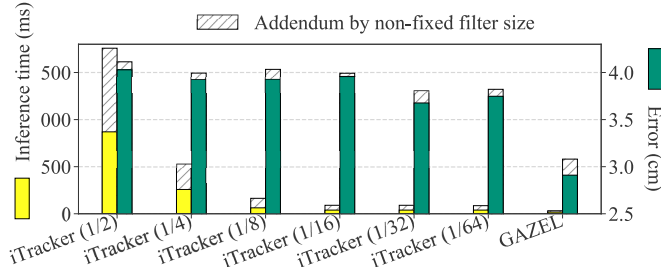


Fig 12. Latency and accuracy comparison for iTracker vs. GAZEL where filter size is fixed to 3×3 (fractions denotes the number of parameters compared to original iTracker).

not consider the filtering process at all. We also checked how well each approach estimates gaze offline. As shown in Table II, GAZEL shows better results than the other methods even before calibration, despite being extremely lightweight compared to them. GAZEL's accuracy is further improved through implicit calibration and filtering. The results in Table II suggest that the overall accuracy of GAZEL is expected to improve 2.01 times compared to the output before calibration. This is an improvement of 1.28 times compared to the existing explicitly calibrated iTracker and 2.59 times compared to the non-calibrated GazeEstimator.

To compare the overhead of GAZEL with previous works, the inference time was measured for iTracker, a method with higher accuracy than GazeEstimator. To conduct a runtime experiment with iTracker, we reimplemented several iTracker models so that the number of trainable model parameters is 1/2, 1/4, 1/8, 1/16, 1/32, and 1/64 of the original model. Kraska et al. [9] did not explain in detail how the CONV layers and FC layers were reduced. We arbitrarily decreased the model size by downsizing the eye crop or the face crop, reducing the number of CNN filters, and adjusting the number of outputs of the FC layer. Fig. 12 shows the results for measuring the accuracy and inference time while reducing the size of iTracker on Galaxy S9. The solid area denotes the case where the filter size is fixed to 3×3 ; the hatched area denotes the additional error when the CONV filter size is not fixed. The cases of 1/16, 1/32, and 1/64, where only the FC layer size is reduced, did not significantly affect the inference speed. However, the 1/2, 1/4, and 1/8 cases, where the amount of CONV computation was reduced by downsizing the size of the input image or the CONV layer, increased the inference speed more than twice. In addition, in all cases, the inference speed was improved approximately two times if the CONV filter size was fixed to 3×3 . The results indicate that the accuracy did not decrease even if the inference speed was improved by reducing the model's size. The design of the GAZEL gaze estimation model builds on this finding. To

speed up the inference speed, we designed the CONV layer of GAZEL to have a fixed filter size of 3×3 and a fixed number of 32 channels. By integrating the Add layer and additional input, GAZEL achieved 27.5% better accuracy than iTracker while the model size was 133 times smaller.

VII. DISCUSSION

In GAZEL, the dataset should be collected under constrained conditions to exploit the tablet-to-smartphone transfer learning technique. In this work, we showed that the tablet data-based model, instead of the model using multiple smartphones, is advantageous in improving the accuracy of gaze tracking on a smartphone. With the present work, however, we were unable to obtain a massive data set that included various lighting conditions and a large number of participants under the specific constraints of head poses. If large-scale data are collected with a specific tablet through crowdsourcing, GAZEL will be utilized more pervasively.

In the current implementation of GAZEL, the face detection latency using the Firebase ML Kit is a bottleneck for the whole framework. The ML Kit detects the face only if the whole face is visible in a frame. However, in actual use cases, the lower part of the face is often cut off, not displaying the full face on the front-facing camera image. Therefore, GAZEL needs a new feature extraction model that is faster than the current ML Kit-based method of finding face landmarks. As part of our future work, we plan to develop a light feature extraction model that can extract Euler angles, eye contour, and face position quickly from an image.

VIII. CONCLUSION

We were motivated to develop GAZEL by observing that the tablet data model is inherently more accurate than the smartphone data model due to the tablet's large-screen form factor. Based on this observation, we proposed tablet-to-smartphone transfer learning that maximizes the model's accuracy by using the tablet data model and transplanting it to a smartphone. A lightweight CNN architecture and implicit calibration were used to enable gaze tracking with high accuracy by reflecting personal characteristics and hardware characteristics at runtime. To the best of our knowledge, GAZEL is the first end-to-end framework to provide a gaze tracker that can run practically on a smartphone.

We believe that the proposed methodology will help create a more advanced smartphone-based gaze-tracking technology. In addition, we expect GAZEL to open up various applications and studies in the smartphone environment that require precise gaze estimation technology.

REFERENCES

- [1] T. Rudnicki. "Eye-control Empowers People with Disabilities." [Online] Available: <https://www.abilities.com/community/assistive-eye-control.html>
- [2] M. Wedel and R. Pieters, "A review of eye-tracking research in marketing," *Review of marketing research*, vol. 4, no. 2008, pp. 123-147, 2008.
- [3] F. Alt, A. S. Shirazi, A. Schmidt, and J. Mennenöh, "Increasing the user's attention on the web: using implicit interaction based on gaze behavior to tailor content," in *Proceedings of the 7th Nordic Conference on Human-Computer Interaction: Making Sense Through Design*, 2012, pp. 544-553.
- [4] M. S. Atkins, G. Tien, R. S. Khan, A. Meneghetti, and B. Zheng, "What do surgeons see: capturing and synchronizing eye gaze for surgery applications," *Surgical innovation*, vol. 20, no. 3, pp. 241-248, 2013.
- [5] D. Novak and R. Riener, "Enhancing patient freedom in rehabilitation robotics using gaze-based intention detection," in *2013 IEEE 13th International Conference on Rehabilitation Robotics (ICORR)*, 2013: IEEE, pp. 1-6.
- [6] M. Khamis, F. Alt, and A. Bulling, "The past, present, and future of gaze-enabled handheld mobile devices: Survey and lessons learned," in *Proceedings of the 20th International Conference on Human-Computer Interaction with Mobile Devices and Services*, 2018, pp. 1-17.
- [7] R. Biedert, A. Dengel, G. Buscher, and A. Vartan, "Reading and estimating gaze on smart phones," in *Proceedings of the symposium on eye tracking research and applications*, 2012, pp. 385-388.
- [8] P. Weill-Tessier, J. Turner, and H. Gellersen, "How do you look at what you touch? A study of touch interaction and gaze correlation on tablets," in *Proceedings of the Ninth Biennial ACM Symposium on Eye Tracking Research & Applications*, 2016, pp. 329-330.
- [9] K. Krafka et al., "Eye tracking for everyone," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 2176-2184.
- [10] L. Jigang, B. S. L. Francis, and D. Rajan, "Free-head appearance-based eye gaze estimation on mobile devices," in *2019 International Conference on Artificial Intelligence in Information and Communication (ICAIIIC)*, 2019: IEEE, pp. 232-237.
- [11] D. W. Hansen and Q. Ji, "In the eye of the beholder: A survey of models for eyes and gaze," *IEEE transactions on pattern analysis and machine intelligence*, vol. 32, no. 3, pp. 478-500, 2009.
- [12] Z. Zhu and Q. Ji, "Eye gaze tracking under natural head movements," in *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05)*, 2005, vol. 1: IEEE, pp. 918-923.
- [13] M. X. Huang, J. Li, G. Ngai, and H. V. Leong, "Screenglint: Practical, in-situ gaze estimation on smartphones," in *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems*, 2017, pp. 2546-2557.
- [14] R. Valenti, N. Sebe, and T. Gevers, "Combining head pose and eye location information for gaze estimation," *IEEE Transactions on Image Processing*, vol. 21, no. 2, pp. 802-815, 2011.
- [15] E. Wood and A. Bulling, "Eyetable: Model-based gaze estimation on unmodified tablet computers," in *Proceedings of the Symposium on Eye Tracking Research and Applications*, 2014, pp. 207-210.
- [16] K.-H. Tan, D. J. Kriegman, and N. Ahuja, "Appearance-based eye gaze estimation," in *Sixth IEEE Workshop on Applications of Computer Vision, 2002.(WACV 2002). Proceedings.*, 2002: IEEE, pp. 191-195.
- [17] F. Lu, Y. Sugano, T. Okabe, and Y. Sato, "Adaptive linear regression for appearance-based gaze estimation," *IEEE transactions on pattern analysis and machine intelligence*, vol. 36, no. 10, pp. 2033-2046, 2014.
- [18] Q. Huang, A. Veeraraghavan, and A. Sabharwal, "TabletGaze: dataset and analysis for unconstrained appearance-based gaze estimation in mobile tablets," *Machine Vision and Applications*, vol. 28, no. 5-6, pp. 445-461, 2017.
- [19] X. Zhang, Y. Sugano, M. Fritz, and A. Bulling, "Appearance-based gaze estimation in the wild," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2015, pp. 4511-4520.
- [20] X. Zhang, M. X. Huang, Y. Sugano, and A. Bulling, "Training person-specific gaze estimators from user interactions with multiple devices," in *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, 2018, pp. 1-12.
- [21] A. E. Hoerl and R. W. Kennard, "Ridge regression: Biased estimation for nonorthogonal problems," *Technometrics*, vol. 12, no. 1, pp. 55-67, 1970.
- [22] A. J. Smola and B. Schölkopf, "A tutorial on support vector regression," *Statistics and computing*, vol. 14, no. 3, pp. 199-222, 2004.
- [23] Y. Zhang and A. J. Hornof, "Easy post-hoc spatial recalibration of eye tracking data," in *Proceedings of the symposium on eye tracking research and applications*, 2014, pp. 95-98.
- [24] Y. Sugano, Y. Matsushita, and Y. Sato, "Appearance-based gaze estimation using visual saliency," *IEEE transactions on pattern analysis and machine intelligence*, vol. 35, no. 2, pp. 329-341, 2012.
- [25] Y. Zhang and A. J. Hornof, "Mode-of-disparities error correction of eye-tracking data," *Behavior research methods*, vol. 43, no. 3, pp. 834-842, 2011.
- [26] F. Alnajjar, T. Gevers, R. Valenti, and S. Ghebreab, "Calibration-free gaze estimation using human gaze patterns," in *Proceedings of the IEEE international conference on computer vision*, 2013, pp. 137-144.
- [27] D. J. Liebling and S. T. Dumais, "Gaze and mouse coordination in everyday work," in *Proceedings of the 2014 ACM international joint conference on pervasive and ubiquitous computing: adjunct publication*, 2014, pp. 1141-1150.
- [28] J. Huang, R. White, and G. Buscher, "User see, user point: gaze and cursor alignment in web search," in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, 2012, pp. 1341-1350.
- [29] M. X. Huang, T. C. Kwok, G. Ngai, S. C. Chan, and H. V. Leong, "Building a personalized, auto-calibrating eye tracker from user interactions," in *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems*, 2016, pp. 5169-5179.
- [30] "Tobii Eye Tracker." [Online] Available: <https://www.tobiipro.com/product-listing/fusion/>
- [31] A. Mariakakis, M. Goel, M. T. I. Aumi, S. N. Patel, and J. O. Wobbrock, "SwitchBack: Using focus and saccade tracking to guide users' attention for mobile task resumption," in *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems*, 2015, pp. 2953-2962.
- [32] Z. Jiang et al., "VADS: Visual attention detection with a smartphone," in *IEEE INFOCOM 2016-The 35th Annual IEEE International Conference on Computer Communications*, 2016: IEEE, pp. 1-9.
- [33] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," *Communications of the ACM*, vol. 60, no. 6, pp. 84-90, 2017.
- [34] "Android Firebase ML Kit." [Online] Available: <https://developers.google.com/ml-kit?hl=ko>
- [35] R. Fares, S. Fang, and O. Komogortsev, "Can we beat the mouse with MAGIC?," in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, 2013, pp. 1387-1390.
- [36] G. Developers. "RenderScript Overview." [Online] Available: <https://developer.android.com/guide/topics/renderscript/compute>
- [37] TensorFlow. "TensorFlow Lite converter." [Online] Available: <https://www.tensorflow.org/lite/convert/index>
- [38] TensorFlow. "Load and run a model in Java." [Online] Available: https://www.tensorflow.org/lite/guide/inference#load_and_run_a_model_in_java